banana-brain: how the brain network works

Date: October 24, 2026 · **Subject**: the raw brain network (`RawBrainNetwork`) and its execution copy (`BrainNetwork`), the logic only (not the code), its settings and the ideas behind it

> **How to read this document**
>
> 1. The idea and how it differs from a layered network.
> 2. The space, the links, the execution in time steps.
> 3. The learning: the score, then the links that are cut and created.
> 4. Every setting and what it changes, then the behaviour seen in the measures.
> 5. The ideas for later: specialized blocks, 3D, neurons that are born, die and move.
>
> The numbers come from the measures of `audit/2026-10-24-brain-training.pdf` . Part 9 is a **proposal**, not implemented.

Contents

1. The idea
2. The space
3. The links
4. The execution
5. The learning
6. Links that are cut and created
7. Settings and what they change
8. Behaviour seen in the measures
9. Ideas for later

1. The idea

A layered network fixes its architecture in advance: every neuron of a layer reads every neuron of the layer before, the signal crosses the network once. The brain network starts from another picture: **neurons spread in a space, linked to their neighbours, a signal that travels for some time, and links that are born and die with what is learned**.

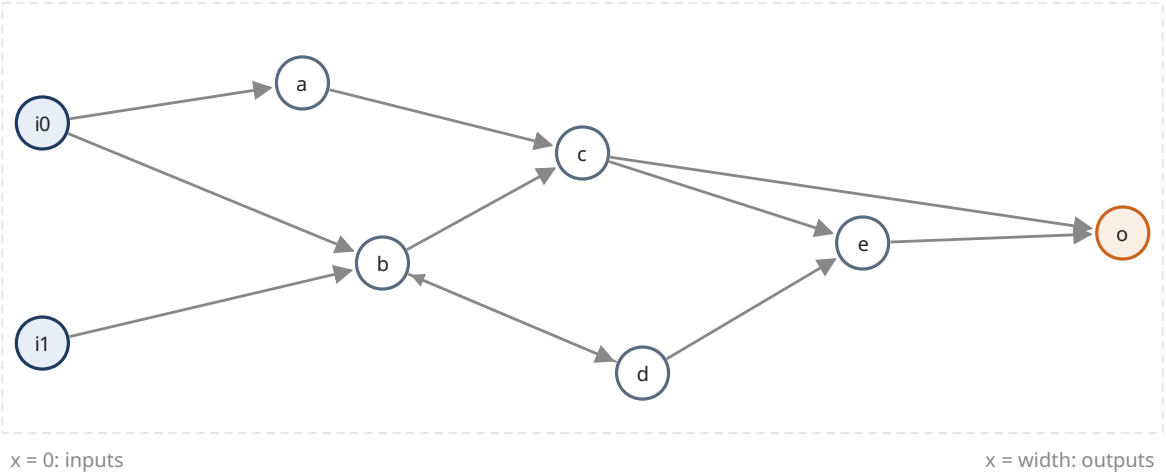
	Layered network	Brain network
Architecture	fixed: sizes of the layers	grows by itself: the links change during the learning
Links	all between two layers	a few per neuron (the cap), to its nearest neurons
Signal	crosses the layers once	moves one link per time step, can loop
Cost	grows with the square of a layer	grows with neurons x cap x steps (sparse)
Learning	descent on the error	a score: what is good is strengthened, what is bad weakened, and the routes change

The goals behind it:

1. **Imitate a brain.** Neurons in space, local links, a signal that propagates in several steps, links that appear and disappear (plasticity).
2. **A topology that organizes itself.** Nothing says which neuron serves what: the network finds its routes by cutting useless links and widening the useful ones.
3. **Sparse and efficient.** A neuron only reads a few links, so a large network stays cheap, and every neuron of a step is computed at the same time (GPU).
4. **Learn what is good and what is not, and reroute.** The score says if an output was good or bad; it changes the weights, and the weights change the links: a weak route disappears, a strong one recruits neighbours.
5. **Specialized blocks (lobes)**, later: zones dense inside and weakly linked to each other, each with its own rules (part 9).

2. The space

Figure 1: the 2D space and the links to the nearest neurons

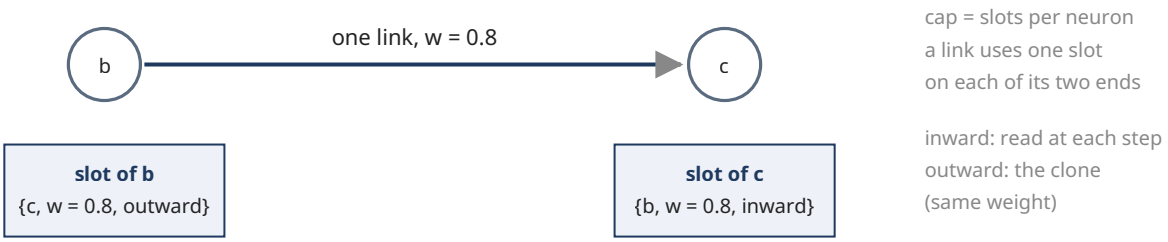


Inputs `i0`, `i1` on the left edge, the output `o` on the right edge, the hidden neurons `a` to `e` placed at random; the dashed link `d -> b` is a loop, possible with random directions.

- The space is a rectangle `width x height`. The **inputs** are spread on the left edge, the **outputs** on the right edge, the other neurons are placed at random inside (the seed of the library chooses where).
- The distance between two neurons decides who is linked to whom: close neurons talk to each other, far ones only through others. The size of the space does not change the links, only the positions relative to each other do.
- The number of neurons is the total: inputs + hidden + outputs.

3. The links

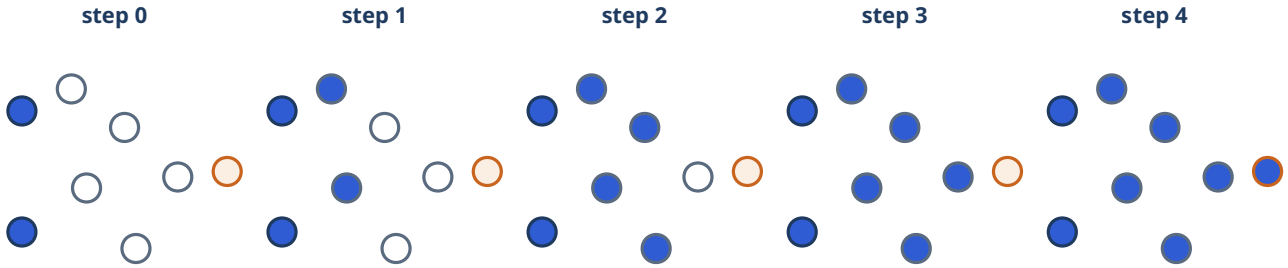
Figure 2: a link is two slots, inward on the receiver, its clone on the sender



1. **The cap.** Each neuron has `cap` slots of links: `min(percentage x n, hook(n))`, by default `min(n, 3 sqrt n)` (12 at 16 neurons, 94 at 1000). It is the only limit of the topology: a neuron never has more links than its slots.
2. **A link and its clone.** A link joins two neurons and has one weight. It is stored twice: inward on the receiver (the one read at each step), outward on the sender (the same weight, used to change the links from both sides).
3. **The init.** Each neuron is linked to as many of its nearest neurons as its cap allows, the closest first, without skipping one unless it cannot be linked (full neuron, two inputs, two outputs). Weights start at 1.0 and biases at 0.0, then `weightInit()` / `biasInit()` draw them at random.
4. **Roles.** An input only sends, an output only receives. Between two hidden neurons the direction is random (loops are possible), or always from left to right with `leftToRight` (no loop: the signal only moves towards the outputs).
5. **Options at the init.** `farLinks(k)` gives `k` slots to random far neurons; `pathInit(h)` adds the missing links so that every input reaches every output in at most `h` links.

4. The execution

Figure 3: the signal moves one link per step (blue: reached)



a neuron reads, at each step, the states of the step before through its inward links: the signal moves one link per step;
the output only depends on the inputs once the number of steps is at least the number of links of the shortest path (here 4)

- The inputs are set, every other neuron starts at 0. At each **time step**, every neuron computes `activation(bias + sum of weight x state of the sender at the step before)` on its inward links. The inputs keep their value.
- After the last step, the outputs are read. The number of steps is `min(n, 4 log2 n)` by default: it must be at least the length of the path from the inputs to the outputs, more lets the signal loop and settle.
- A **signal threshold** (`threshold`) sets to 0 a state too small to matter, so a weak signal stops instead of spreading noise.
- The **execution copy** (`BrainNetwork` , the reduced network) keeps only what the execution needs: the biases and the inward links packed. It cannot learn; `reduce()` gives it, a file `.bn` stores it, the raw network and its positions are in the `.rbn` .

5. The learning

1. **The score.** After an execution, each output gets a score: positive = what it did was good, negative = bad. With a known target it is `rate x (target - output)` (what `train()` sends); without a target it can be any reward. The score is not a loss to minimize: it is a signal to follow (an ascent).
2. **Back in time.** The score goes back through the steps of the last execution: an output gives it to the links it read at the last step, scaled by how much each one counted (weight, slope of the activation), and so on step after step. A link that carried a useful signal at the right time gets stronger, one that pushed the wrong way gets weaker; the biases follow the same rule.
3. **Then the topology.** Once the weights moved, the links at a limit change (part 6). The learning changes both the strength and the shape of the network.

Options of the learning (all off by default):

Option	What it does
`momentum(m)`	each step keeps `m` of the step before: smoother, faster on small tasks
`adam(rate)`	each weight moves by about the rate, the direction from the mean of its gradients: the size of the score no longer matters (the best measured: CIRCLE 84 %)
`revive()`	at the end of a round, a neuron stuck at 0 or 1 gets its bias at 0 and its inward weights halved
`stepLimit(l)`	no weight moves by more than `l` at once: no jump
`batch(true)` + `applyBatch()`	the scores of several samples are averaged and applied once: no zigzag
`clampWeights(true)`	no weight or bias above `weightMax`
`directOutput(true)`	the score reaches the output without the slope of its activation: a saturated output still learns

6. Links that are cut and created

After each learning step, every neuron looks at its links, the nearest first, as they were before any change of this round:

1. **Too weak: cut.** A link whose absolute weight is at `weightMin` or under is removed with its clone: it carries nothing useful. It stays when it is the last link of one of its two neurons, the last inward link of a hidden neuron or of an output, or the last outward link of an input (unless `allowOrphans()`), and with `keepPaths` when it is the last path of an input to an output.
2. **Too strong: recruit.** A link at `weightMax` or over says "this route matters": the neuron links its nearest neuron that is still free (a free slot on both sides, not linked yet, the direction allowed). The new link gets a random sign and a weight between half and all of the mean of the two nearest links that are neither too weak nor too strong (so it looks like its neighbours), always inside the limits; with `lowNewLinks` it starts small (5 % of the limits) so it does not shake the outputs.
3. **Nothing else moves.** A link between the two limits only changes its weight. A full neuron cannot recruit: its strong links keep growing (part 8).

> **Key takeaway**
> The weights decide the shape. A route that is not used fades and disappears; a route that is used a lot widens by recruiting the neighbours, so the network reroutes itself around what is learned.

7. Settings and what they change

Setting	Default	Higher	Lower
Neurons	given	more routes, more cost, more risk that an input has no path	too few routes for a hard task
Size of the space	1 x 1	nothing by itself (only the relative positions matter)	nothing by itself
Links cap	`min(n, 3 sqrt n)`	more routes, but a big sum saturates the sigmoid (cap n - 1: 80 % frozen at 60 hidden)	1 link: nothing learns; few: the inputs lose their path
Steps	`min(n, 4 log2 n)`	longer loops, more cost	under the path length the output ignores the inputs
`weightMin`	0.01	cuts more, the network gets sparse faster (can cut useful routes)	almost never cuts
`weightMax`	2.0	recruits later, strong links stay alone	recruits a lot, the network gets dense fast
`threshold`	0	silences weak signals: the blocks talk less	every signal spreads
Direction	random	`leftToRight` : no loop, a feed-forward network shaped by space	random: loops, memory over the steps
Score rate	given	jumps, zigzag, frozen outputs (rate 10)	slow but steady (rate 1 - 3)
`weightInit` range	the random range	big sums at once, saturation early	small signals, slow start
`allowOrphans`	off	a neuron can end alone (dead)	every neuron keeps a link

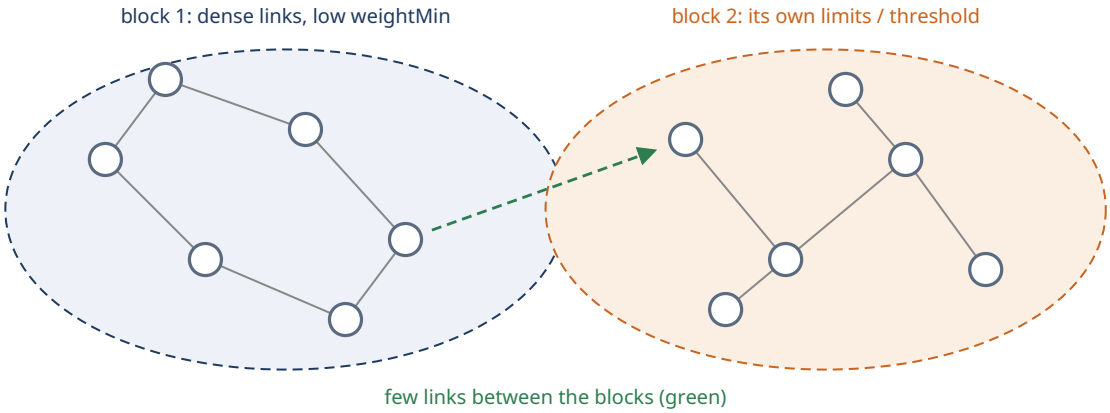
8. Behaviour seen in the measures

1. **All or nothing.** A network finds the right routes or stays on a wrong solution (XOR: 3 samples of 4 right); training longer does not help, another seed does.
2. **Saturation.** When a neuron is full, its strong links cannot recruit and keep growing; the sum gets huge, the sigmoid is flat, the gradient is 0 and the network freezes (flat error). This is the main failure left on CIRCLE and on the curves.
3. **Lost paths.** The cuts can remove the last route of an input: the output then depends on fewer inputs and the task cannot be solved; `keepPaths` and the filter on every input prevent it.
4. **Distance matters.** With local links, a far input needs many steps and many links in a row; the signal fades at each sigmoid (about x 0.25 for the gradient), so the shortest path counts more than the number of neurons.
5. **Best measured settings.** The default caps with Adam 0.01, `pathInit`, `keepPaths`, filter on every input and low new links: XOR 96 %, AND 100 %, CIRCLE 84 % (88 % with `revive`). Large networks: the new caps (AND 95 % at 120 hidden).

9. Ideas for later

9.1 Specialized blocks (lobes)

Figure 5: specialized blocks (lobes)



A brain has zones that do one thing, dense inside and linked to each other by a few routes. The network already tends to it (links are local), the idea is to make it explicit:

Idea	What it gives
Zones of the space with their own limits (`weightMin` , `weightMax` , threshold, cap)	a zone that keeps its links, another one that prunes fast
A higher `weightMin` for the links between two zones	the zones get separated unless a route between them is really used
A threshold per zone	a zone only wakes up on a strong signal: it specializes on what reaches it
Inputs / outputs tied to a zone	each zone learns its own part of the task
Score per zone	a zone is rewarded only for its own outputs

9.2 3D space

The same rules in 3D: each neuron has more neighbours at the same distance, the paths are shorter (about `n^(1/3)` links across instead of `n^(1/2)`), so fewer steps for the same number of neurons. The distances, the nearest lists and the path search only need a third coordinate.

9.3 Neurons that are born and die

Event	Rule (proposal)	Effect
Death	a neuron without link, or always at 0, is removed	the network loses what it does not use
Birth	a new neuron where links are full and strong (a crowded route)	more capacity where the task needs it, rescues a saturated neuron

9.4 Neurons that move

A neuron moves towards the neurons it is strongly linked to and away from the weak ones: the useful groups get closer, their nearest lists change, and the new links follow the routes that work. It is the step that turns the space itself into something learned, and a natural way to form the blocks of 9.1.

> **Key takeaway**
> Today the network learns its weights and its links; birth, death and movement would let it also learn its size and its geometry.